

Supplementary Material for “Correct but Slow: An Empirical Study of the GPU Kernel Evaluation Gap in Modern Domain-Specific Languages”

Anonymous Author(s)
Anonymous Institution

1 **Scope.** This document collects the appendices of the main
2 paper: threats to validity (appendix A), the cross-architecture
3 generalization study (appendix B), the per-kernel library-
4 efficiency table (appendix C), and reproducibility details
5 (appendix D). Lettered appendices and “S”-numbered floats
6 (e.g. Table S1, Fig. S1) refer to this document; all other cross-
7 references (e.g. Section IV, Table I, Fig. 1) resolve into the
8 main paper. Citations refer to this document’s own reference
9 list.

10 APPENDIX A 11 THREATS TO VALIDITY

12 **Construct validity.** Our primary metric, library efficiency,
13 measures DSL performance relative to the strongest library
14 baseline we were able to exercise for each kernel. Because
15 cuBLAS and cuDNN select among multiple internal algorithms,
16 any failure to invoke their best-performing configuration would
17 make the DSL appear closer to library performance than it
18 actually is. To reduce this risk, we use library-backed PyTorch
19 paths consistent with standard practice, invoke `cudaFind`
20 for convolution, record the selected algorithms, and allow
21 cuBLAS to use a large workspace. We report under default
22 cuDNN flags (no explicit `cuda.benchmark` toggle, no
23 NHWC conversion); convolutions run in the PyTorch-default
24 NCHW layout, and algorithm-selection variance is bounded
25 by `cudaFind` defaults.

26 **Internal validity.** Profiling-based analysis may be affected
27 by measurement noise and tool overhead. We mitigate this
28 threat by separating end-to-end timing from counter collection,
29 using repeated runs for both, and verifying that key Nsight
30 Compute measurements remain stable across repetitions. All
31 experiments are executed in isolation on a fixed software and
32 hardware setup to reduce confounding effects from concurrent
33 workloads or environmental variation. Locked-clock re-timing
34 (graphics 1215 MHz / memory 1215 MHz) over 100 runs shows
35 run-to-run relative std-dev of 0.0–0.9% on near-parity kernels,
36 so the reported gaps lie far outside measurement variation
37 rather than being artifacts of frequency variation.

38 **Auto-tuning shape sensitivity.** Our auto-tuning sweep
39 selects each kernel’s configuration by timing the candidate grid
40 at a fixed per-kernel shape that, for several kernels, is smaller

than the large deployment shape used for benchmarking. At
the smaller shape the candidates often fall within measurement
noise, so the selected configuration can be effectively arbitrary
and is occasionally slower than the implementation’s hard-
coded default at the deployment shape (we observed Triton
`mean_reduction` at +136% and `batched_matmul` at
+54% relative to the default). We re-tuned the kernels where
this occurred (`batched_matmul`, `mean_reduction`, and
`attention` in Triton) at the deployment shape, restoring
configurations at or below the default (1.3–3.0× faster than
the small-shape selection); this touches only those auto-tuned
rows and does not affect the reported root causes or heuristics.

External validity. Our study covers 22 kernels across
five operator categories, but it does not represent the full
design space of GPU workloads. In particular, the evaluated
configurations emphasize common deep learning operators
and shapes rather than uncommon cases such as highly
dilated convolutions, extreme group counts, or very small
batches. Conv2d is evaluated across 1×1, 3×3, 5×5, 7×7,
depthwise, and strided variants. In addition, our primary
measurements span two architectures—the A100-SXM4-40GB
(Ampere, `sm_80`) and the GH200 (Grace Hopper, `sm_90`)—
with further cross-architecture replay on the A100-PCIE-40GB
(same `sm_80`, a form-factor robustness check) and H100-80GB-
HBM3 (a cross-family generalization check); results on other
vendor backends may still differ.

Conclusion validity. Our root-cause claims are based on a
combination of performance measurements, hardware-counter
evidence, and targeted mitigations. Although the mitigation
results strengthen the causal interpretation, they do not rule out
additional compiler or runtime factors that were not isolated
in this study. The conclusions should therefore be read as
evidence-supported explanations for the observed gaps, rather
than as an exhaustive account of all possible causes.

The suite is a probe, not a proposed benchmark. Our 22-
kernel suite is deliberately not offered as a comprehensive per-
formance benchmark—a central claim of this work (section IV)
is that such a benchmark is combinatorially infeasible to build
and maintain. The suite instead functions as a diagnostic
probe: it is large enough to expose the evaluation gap (correct
kernels that existing benchmarks admit yet run far below the

TABLE S1

Cross-architecture generalization. Median library efficiency E_{lib} (%) per kernel category for each DSL on the primary A100-SXM4 (sm_80) and the cross-architecture replays A100-PCIE (sm_80, form factor) and H100 (sm_90, Hopper). All cells are *untuned defaults*, frozen from the initial pre-tuning sweep as a like-for-like robustness check; they therefore differ from the tuned main-paper profile (table I). In particular, the TileLang convolution medians (7.8–11.1%) reflect the untuned configuration diagnosed as a tuning artifact in section IV (tuned: 59.3% on the A100), and the Triton normalization medians include the `rms_norm` unfused-baseline artifact that table I excludes. The qualitative profile is preserved across architectures: GEMM and element-wise competitive, Triton convolution and TileLang normalization severely behind.

Category	Triton			TileLang		
	SXM4	PCIE	H100	SXM4	PCIE	H100
GEMM	77.3%	75.5%	68.4%	69.4%	60.3%	69.2%
Convolution	31.6%	38.0%	37.7%	9.5%	11.1%	7.8%
Normalization	121.5%	137.3%	130.5%	1.0%	0.8%	0.9%
Element-wise/Reduction	65.3%	73.0%	72.9%	51.0%	55.3%	60.9%

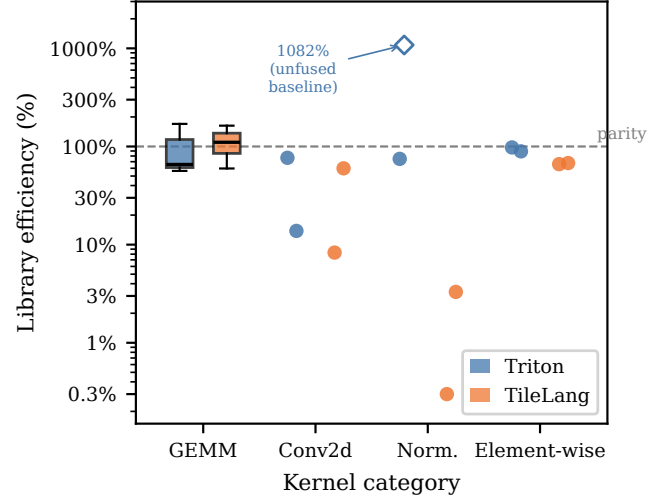


Fig. S1. Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN on the NVIDIA GH200-480GB, by kernel category (log scale; dashed line = 100% parity). Same layout as Figure 1: GEMM shows IQR boxes; other categories show per-kernel dots. $\diamond$ 1082% (Triton, Norm.) marks `rms_norm`: unfair-baseline artifact, not a DSL win. TileLang Conv2d is competitive on both (GH200 59.9% large; A100 $\sim$ 59%); TileLang Norm. remains collapsed (0.3%). Attention excluded (Section IV-C).

TABLE S2

Root-cause taxonomy reproduces across architectures. Each corrected root cause measured on all three GPUs with the identical portable harness. All reproduce, confirming they are properties of the DSL/compiler rather than of a single GPU (cf. table S1).

Root cause	SXM4	PCIE	H100	Reproduces
RC0 TileLang LayerNorm anomaly (8192 ²) E_{lib}	0.09%	0.09%	0.08%	yes (memory-latency bound)
Register-spill check on Triton conv (n_{spills} ; RC3 does not extend to conv)	0	0	0	yes (no spill; occupancy-bound)
RC4 Winograd upper-bound contribution	0.1%	2.0%	20.7%	yes ($\sim$ 0–2%; not primary)
FP32 $\rightarrow$ q_{norm} TF32 lowering (rel. err)	633 $\times$	633 $\times$	1268 $\times$	yes (TF32 mantissa class)

[†] The cuDNN deterministic-mode (Winograd-off) timing on H100 is high-variance ($\sigma \approx 27\%$ of median, un-locked clocks), so its determinism A/B over-states the Winograd upper bound; the stable, locked-clock A100-SXM4 and A100-PCIE measurements bound the Winograd contribution at $\leq 2\%$.

APPENDIX B

CROSS-ARCHITECTURE GENERALIZATION

The main-paper tables report results on the two primary architectures (A100-SXM4 and GH200). As a robustness check, we replay the kernel suite and the root-cause taxonomy along an A100-SXM4 / A100-PCIE / H100 axis: the A100-SXM4 serves as the anchor, the A100-PCIE-40GB is a same-architecture (sm_80) form-factor control, and the H100-80GB-HBM3 is a cross-family (sm_90) control. The GH200 primary results remain in the main-paper tables.

Figure S1 shows the per-category library-efficiency distribution on the GH200, parallel to Figure 1 in the main paper. The GH200 profile is consistent with the A100: TileLang convolution is competitive on both (GH200 small-shape 8.3% and large-shape 59.9%; A100 7.2% and 59.0%), while TileLang normalization remains collapsed on both (LayerNorm 0.3%). This confirms that the normalization gap is an authoring artifact independent of architecture, while the residual convolution gap is the Triton arm, which holds on both architectures (Section V).

APPENDIX C

PER-KERNEL LIBRARY EFFICIENCY

Table S5 lists $E_{\text{lib}} = t_{\text{PyTorch}}/t_{\text{DSL}}$ for each kernel at the large input shape on the A100-SXM4, tuned configurations. Values below 100% indicate the DSL is slower than the library baseline. Four near-parity element-wise kernels (`leaky_relu`, `swiglu`, `logsumexp`, `cross_entropy`) exceeded 100% on both DSLs at the large shape and are omitted from this table; their main-paper medians appear in Table I. With those four, the table accounts for all 22 suite kernels.

APPENDIX D

REPRODUCIBILITY DETAILS

A. Kernel Suite Provenance

a) *Selection criteria:* We narrow the candidates from TritonBench [1] and the TileLang example repository [2] using four criteria applied in order. (i) *Forward-pass only:* backward and gradient kernels are excluded, as they are governed by different access patterns and reduction structures.

TABLE S3

GEMM autotuning recovery generalizes (RC2). Triton matmul E_{lib} before (plain, heuristic) and after (expanded autotune search) on all three GPUs. The default-grid-versus-expanded-search gap (section IV-C vs. section VI-D) is a search-space artifact on every architecture, not shape- or hardware-specific.

Shape	Triton plain			Triton autotuned		
	SXM4	PCIE	H100	SXM4	PCIE	H100
4096 ²	56.4%	57.7%	43.2%	76.5%	110.1%	33.1%
16384 ²	32.5%	38.3%	33.6%	81.8%	87.5%	70.2%

At the RQ1 16384² shape, expanded autotuning recovers Triton on all three GPUs (plain→autotuned). The sub-millisecond 4096² cells on the un-locked PCIE/H100 replays carry higher relative noise; the locked-clock A100-SXM4 primary is the reference measurement.

TABLE S4

Convolution filter sweep across architectures (large shape 32×256×128², groups=1, stride=1). Triton E_{lib} vs cuDNN for each GPU, with the measured Triton register-spill count. The gap widening with filter size and the absence of register spilling both hold across architectures.

Filter	Triton E_{lib}			Triton n_{spills}
	SXM4	PCIE	H100	
1 × 1	28.7%	37.0%	24.7%	0
3 × 3	19.3%	24.4%	11.4%	0
5 × 5	13.7%	16.9%	13.4%	0
7 × 7	9.8%	12.5%	15.7%	0

(ii) *Stable library baseline*: each kernel must admit a well-defined cuBLAS, cuDNN, or PyTorch eager reference that computes the same function, making library efficiency a meaningful quantity. (iii) *Five-category coverage*: we retain kernels that populate the five operator categories (GEMM, attention, convolution, normalization, element-wise/reduction) rather than over-sampling any single class. (iv) *Canonical operators*: kernels with sparse inputs or runtime-dependent output shapes are excluded for lacking a stable baseline.

b) *Per-kernel provenance*: Table S6 lists all 22 kernels. Triton implementations are drawn from TritonBench except `layer_norm`, which follows the TorchInductor reference implementation. All TileLang implementations are our re-implementations of the same operators. PyTorch implementations are cuBLAS, cuDNN, or PyTorch-eager baselines as described in section D-B.

B. Baseline Specifications

GEMM baselines use cuBLAS via `torch.matmul`. Convolution baselines use PyTorch `nn.Conv2d` with default NCHW memory layout under standard cuDNN algorithm selection. Normalization baselines use `F.layer_norm` and `F.rms_norm`. Element-wise and reduction baselines use standard PyTorch eager execution. For attention, we use PyTorch scaled dot-product attention with the FlashAttention backend enabled through `enable_flash_sdp(True)`.

a) *GEMM notation and exclusions*: Per-shape GEMM latencies appear in Table S5; 16384² denotes a square 16384 × 16384 FP16 GEMM and 64 × 128² a batched GEMM of batch 64 over 128 × 128 matrices. FP32 TileLang GEMM is excluded: `T.gemm` silently lowers to TF32, making FP32 results a

TABLE S5

Per-kernel library efficiency E_{lib} (%) at the large input shape (A100-SXM4, tuned). $E_{\text{lib}} < 100\%$ = DSL slower than PyTorch. Annotated values above 100% reflect baseline artifacts, not DSL speed advantages (see footnotes).

Kernel	Input shape	Triton	TileLang
<i>GEMM</i>			
matmul	16384 × 16384 FP16	59.7%	78.1%
batched_matmul	128 × 2048 × 2048 FP16	97.9%	94.2%
linear_activation	(1, 2048, 4096) FP16	185.5%	468.5% [†]
<i>Attention (excluded from main comparison; see Section IV-C)</i>			
attention	QKV (8, 32, 2048, 128) FP32	7691% [‡]	54.3%
<i>Convolution</i>			
conv2d	32 × 256 × 128 ² , 3 × 3 FP16	28.6%	59.0%
<i>Normalization</i>			
layer_norm	8192 × 8192 BF16	90.2%	0.3%
rms_norm	8192 × 8192 FP16	873% [§]	3.2%
<i>Element-wise / Reduction</i>			
add	64M FP16	90.8%	74.5%
mul	64M FP16	69.5%	67.1%
relu	16384 × 16384 FP16	95.7%	68.1%
softmax	(4096, 32768) FP16	117.2%	0.9%
log_softmax	(4096, 32768) FP16	45.3%	3.6%
argmax	(8192, 32768) FP16	25.5%	5.9%
max_reduction	(8192, 32768) FP16	109.6%	67.2%
mean_reduction	(8192, 32768) FP32	92.7%	97.4%
matrix_transpose	16384 × 16384 FP16	342.8%	67.4%
embedding	(131072, 1024) idx FP16	447.9% [¶]	57.9%
index_select	(65536, 2048) idx FP16	59.2%	56.6%

[†] TileLang `linear_activation` uses a fused gate-and-activate pass; the speedup is genuine. [‡] Triton `attention` uses a FlashAttention-style tiled kernel against PyTorch's unfused eager path; the comparison is excluded from the main GEMM analysis (Section IV-C). [§] Triton `rms_norm` is faster than PyTorch's unfused `F.rms_norm` reference; this is a baseline-choice artifact (Table I footnote). [¶] Triton `embedding` is faster than PyTorch's scatter-based eager lookup; attributed to coalesced vs. scattered memory access, not a library maturity gap. ^{||} Triton `matrix_transpose` is faster than PyTorch's eager transpose-copy path; a baseline-path effect analogous to embedding, not a library maturity gap.

precision artifact rather than a logic error (Section III-C). Table I reports the category medians; the 16384² Triton gap to 59.7% reflects an under-populated auto-tune search space (RC2), while TileLang reaches 78.1% on that shape via an explicit schedule.

C. Measurement Protocol

a) *End-to-end timing*: We measure host-synchronized GPU execution time using `time.perf_counter()` bracketed by `torch.cuda.synchronize()` calls. Each measurement uses 10 warm-up iterations followed by 100 timed iterations; we report the median. Results reflect GPU-side execution only, excluding host-side dispatch overhead, and each workload runs in isolation.

b) *Clock-lock settings*: On the A100-SXM4-40GB, we pin graphics and memory clocks to 1215 MHz and 1215 MHz respectively via `nvidia-smi --lock-gpu-clocks` and `--lock-memory-clocks`. The nominal maximum is 1410 MHz, but that setting power-caps under the 400 W board limit and causes clock throttling; 1215 MHz is stable and yields run-to-run relative standard deviation of 0.0–0.9% across 100-iteration timing windows. On the GH200, clocks are locked at 1320 MHz (graphics) and 2619 MHz (memory).

c) *Nsight Compute counter definitions*: We collect the following seven counters per kernel; each is gathered from

TABLE S6
Per-kernel provenance for the 22-kernel ViperBench suite.

Kernel	Category	Triton Source	TileLang Source
matmul	GEMM	TritonBench	Custom
batched_matmul	GEMM	TritonBench	Custom
linear_activation	GEMM	TritonBench	Custom
attention	Attention	TritonBench	Custom
conv2d	Convolution	TritonBench	Custom
layer_norm	Normalization	TorchInductor ref.	Custom
rms_norm	Normalization	TritonBench	Custom
add	EW / Red.	TritonBench	Custom
mul	EW / Red.	TritonBench	Custom
relu	EW / Red.	TritonBench	Custom
leaky_relu	EW / Red.	TritonBench	Custom
softmax	EW / Red.	TritonBench	Custom
log_softmax	EW / Red.	TritonBench	Custom
logsumexp	EW / Red.	TritonBench	Custom
swiglu	EW / Red.	TritonBench	Custom
argmax	EW / Red.	TritonBench	Custom
max_reduction	EW / Red.	TritonBench	Custom
mean_reduction	EW / Red.	TritonBench	Custom
cross_entropy	EW / Red.	TritonBench	Custom
matrix_transpose	EW / Red.	TritonBench	Custom
index_select	EW / Red.	TritonBench	Custom
embedding	EW / Red.	TritonBench	Custom

“Custom” denotes our re-implementation of the operator following the same interface as the TritonBench version; “TorchInductor ref.” denotes an implementation modelled on the TorchInductor generated reference for that op. EW / Red. = element-wise or reduction.

a single `ncu` execution and verified within 2% across five repeats.

- 1) **Global-load efficiency** — fraction of global memory transactions that serve requested bytes (1.0 = perfectly coalesced).
- 2) **Sectors per request** — average number of L1/L2 cache sectors accessed per memory instruction; lower values indicate better coalescing.
- 3) **Registers per thread** — register file allocation per thread; high values increase occupancy pressure and can trigger spill.
- 4) **Register spills** — bytes spilled from the register file to thread-local memory (L1/L2/DRAM); non-zero values denote register-pressure-induced latency.
- 5) **Long-scoreboard stall cycles** — warp stall cycles waiting for L2 or DRAM data; the dominant stall for memory-bound kernels.
- 6) **Barrier stall cycles** — warp stall cycles at thread-block synchronization barriers (`__syncthreads / bar.sync`).
- 7) **L2 sector hit rate** — fraction of L2 cache sector accesses that hit; low rates indicate DRAM pressure.

d) *KernelBench evaluator harness*: For the passes-but-slow demonstration (section IV-B), we invoke KernelBench’s `eval_kernel_against_ref` function, which (i) overrides the candidate kernel’s `get_inputs` and `get_init_inputs` with those of the reference implementation so input shapes cannot be altered by the candidate, (ii) checks output equivalence on randomized inputs at the same per-dtype tolerances as our correctness suite (section III-C), and (iii) emits a warning only if the candidate’s throughput

exceeds $10\times$ the reference—a speedup-only guard that admits any slowdown. We record pass/fail and measure slowdown separately using our clock-locked timing harness.

D. Full Hardware and Software Stack

TABLE S7
GPU hardware specifications for the two primary architectures.

Property	A100-SXM4-40GB	GH200
Architecture	Ampere (sm_80)	Grace Hopper (sm_90)
SMs	108	132
Memory	40 GB HBM2e	96 GB HBM3
Peak mem. BW	~1.5 TB/s	~4.0 TB/s
Peak FP16 TC	~312 TFLOP/s	~989.5 TFLOP/s
L2 cache	40 MB	60 MB
TDP	400 W	900 W (SoC)
Clock lock (g/m)	1215 / 1215 MHz	1320 / 2619 MHz
NVIDIA Driver	610.43.02	n/r*
CUDA Toolkit	12.8	12.8

* The GH200 node’s driver version was not exported by the measurement harness logs; the CUDA runtime and all library versions are pinned by the artifact’s lockfiles.

a) Hardware:

TABLE S8
Pinned software versions for all experiments.

Package	Version
PyTorch	2.8.0+cu128
Triton	3.4.0
TileLang	0.1.6.post1
cuDNN	bundled with PyTorch
Nsight Compute	2026.2.0

b) *Software stack*: All experiments use default cuDNN algorithm selection flags and pre-warm the Triton auto-tuner before measurement. A `requirements.txt` pinning this framework stack is distributed with the artifact; the pinned PyTorch wheel bundles its own CUDA runtime and cuDNN.

E. Optimization Trajectory: TileLang LayerNorm

To illustrate the correctness-gated iterative protocol of section III-D, table S9 lists the full 18-iteration search for the TileLang LayerNorm kernel of section VI-B. Two edits dominate: iteration 1 replaces the manual `T.serial` reduction with a single `T.reduce` (RC0), and iteration 14 switches to native `bfloat16` I/O, removing the intermediate `.float()` casts. The intervening iterations confirm that thread count, tiling, and reduction algebra are second-order once the kernel is bandwidth-bound, and two configurations fail to compile—evidence that the search is genuine rather than a curated path. Runtimes are on the separate development GPU (section VI); the selected final kernel is re-timed on the A100-SXM4 in table IV.

TABLE S9

TileLang LayerNorm optimization trajectory (development GPU). Full 18-iteration correctness-gated search for the bfloat16 LayerNorm kernel of section VI-B; every listed configuration is numerically correct except the two compile failures (iter 3, 7, shown “—”). E_{lib} is the latency ratio to PyTorch `F.layer_norm` on the same GPU. **Bold** marks the two dominant edits: `T.serial`→`T.reduce` (iter 1) and native bfloat16 I/O (iter 14). [†] selected final configuration; re-timed on the A100-SXM4 as 0.270 ms / 124% in table IV.

Iter	Change	Runtime (ms)	E_{lib}
base	<code>T.serial</code> reduction	1090	0.08%
1	Replace <code>T.serial</code> with <code>T.reduce</code>	5.24	17.1%
2	Register copy, 256 threads	5.22	17.2%
3	2D <code>block_M=4</code> , <code>reduce_sum dim 1</code>	—	—
4	2D <code>T.copy</code> fix, <code>reduce_sum dim 1</code>	5.22	17.2%
5	512 threads	5.22	17.2%
6	fp16 I/O, fp32 accumulate	3.02	29.5%
7	2D merged $E[x^2]-E[x]^2$	—	—
8	1024 threads, fp16 I/O	3.02	29.6%
9	Tiled reduction <code>block_N=1024</code>	2.99	29.9%
10	Two-pass tiled (sum+sq in pass 1)	3.00	29.8%
11	$E[x^2]-E[x]^2$, single load	3.02	29.6%
12	Disable warp-specialized <code>pass_configs</code>	3.02	29.6%
13	Store $x-\mu$ in fp16 to save registers	3.02	29.6%
14	Native bfloat16 I/O, no cast	0.893	99.8%
15 [†]	<code>out_idx</code> output allocation	0.891	100.0%
16	Full fp32 <code>row_copy</code> (more regs)	0.905	98.5%
17	Restore iter-15 configuration	0.893	99.8%
18	128 threads	0.895	99.6%

REFERENCES

- [1] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, H. Wang, J. Wang, X. Han, Z. Liu, and M. Sun, “TritonBench: Benchmarking large language model capabilities for generating triton operators,” in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 23 053–23 066. [Online]. Available: <https://aclanthology.org/2025.findings-acl.1183/>
- [2] L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, F. Yang, and Z. Yang, “Tilelang: Bridge programmability and performance in modern neural kernels,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=Jb1WkNSfUB>